KELOMPOK 3 · PEMROGRAMAN BERORIENTASI OBJEK

Penjelasan Project & Pembagian Materi Presentasi

Sistem Manajemen Data Pasien Penyakit Mata

Penerapan Class & Object, Enkapsulasi, dan Inheritance di Python dengan data nyata pasien poli mata dari dataset ODIR-5K (Ocular Disease Intelligent Recognition).

3.358	7	10	9
pasien (data nyata)	kelompok penyakit mata	file kode (9 class + main.py)	pembicara

Pembagian materi anggota Kelompok 3

No	Nama	Bagian yang dijelaskan	Slide	Durasi
1	**Riswan**	Latar Belakang & Struktur Project	1–5	± 3,5 menit
2	**Yus Aidil**	Dataset ODIR-5K & Pemetaan Kolom	6–9	± 3,5 menit
3	**Fahmi**	Class & Object	10–13	± 3,5 menit
4	**Diva Nurislamia Agfa**	Enkapsulasi	14–18	± 4 menit
5	**Rosa Ramadani**	Inheritance: Pewarisan Bertingkat	19–22	± 3 menit
6	**Sofiang**	Komposisi: Pemeriksaan Mata	23–26	± 3 menit
7	**Revalinda**	Hierarki Class Penyakit & Polimorfisme	27–30	± 3,5 menit
8	**Andi Rezky Nabila**	Program Utama: Membaca CSV & Menu	31–34	± 4 menit
9	**Alfira Yunita Syam**	Hasil Program, Demo Menu & Kesimpulan	35–41	± 5 menit

Total durasi ± 33 menit, dihitung dari panjang naskah.

Cara memakai dokumen ini

- Bagian **Gambaran Project** menjelaskan struktur folder, file, data, diagram, dan letak konsep OOP. Semua anggota sebaiknya membacanya.
- Lalu setiap anggota membaca **bagiannya sendiri**: inti yang disampaikan, kode yang ditunjukkan beserta fungsi setiap barisnya, output program, dan persiapan tanya jawab.
- Nomor baris kode **sama persis** dengan file di folder `program`. Naskah lisan lengkap ada di folder `naskah`.
- Semua angka dan output diambil dari program yang **benar-benar dijalankan**, bukan contoh karangan.

GAMBARAN PROJECT

Struktur, data, diagram, dan letak konsep OOP

Tujuan & sumber data

- Program mengelola data pasien poli mata: membaca CSV, mengubah setiap pasien menjadi object, memeriksa setiap nilai dengan setter, lalu menyediakan menu untuk melihat, mencari, menambah, dan merangkum data.
- Data: **ODIR-5K (Ocular Disease Intelligent Recognition)**, 6.392 baris (1 baris = 1 foto fundus) dari 3.358 pasien. Sumber: https://www.kaggle.com/datasets/andrewmvd/ocular-disease-recognition-odir5k
- Program ini mengelola data diagnosis dari dokter, bukan alat diagnosis. Data hanya memuat penyakit yang terlihat di foto fundus.

Struktur folder project

```
Tugas Kelompok/
├── program/        → kode Python (10 file) + data/full_df.csv
├── ppt/            → presentasi HTML + versi PDF
├── naskah/         → naskah presentasi (Word + PDF)
├── penjelasan/     → dokumen ini (pembagian materi)
├── laporan/        → laporan Word + PDF
├── arsip/          → versi lama (data rumah sakit umum)
├── README.md       → catatan struktur & cara menjalankan
└── PROMPT.md       → prompt final project
```

Isi folder program

File	Isi
`main.py`	Program utama: membaca CSV, demo validasi, menu interaktif
`orang.py`	class Orang: umur, jenis_kelamin
`pasien.py`	class Pasien(Orang): id_pasien
`pasien_mata.py`	class PasienMata(Pasien): mata kiri/kanan, kode diagnosis, daftar penyakit
`pemeriksaan_mata.py`	class PemeriksaanMata: hasil foto satu mata
`penyakit_mata.py`	class PenyakitMata: satu jenis penyakit mata
`retinopati_diabetik.py`	class RetinopatiDiabetik(PenyakitMata)
`degenerasi_makula.py`	class DegenerasiMakula(PenyakitMata)
`glaukoma.py`	class Glaukoma(PenyakitMata)
`kelainan_lain.py`	class KelainanLain(PenyakitMata)
`data/full_df.csv`	data asli ODIR-5K (6.392 baris)

Cara menjalankan

- Buka terminal di folder project, masuk ke folder `program`, lalu jalankan `python main.py`. Hanya memakai library bawaan Python (`csv` dan `os`).

```
> python main.py
================================================================
SISTEM MANAJEMEN DATA PASIEN PENYAKIT MATA
================================================================
File data          : full_df.csv
Pasien dimuat      : 3358 object PasienMata
Baris digabung     : 3034 (foto mata kedua dari pasien yang sama)
Baris dilewati     : 0
```

Kolom CSV menjadi atribut

Kolom CSV	Class	Atribut	Aturan (setter)
`ID`	Pasien	`id_pasien`	bilangan bulat ≥ 0
`Patient Age`	Orang	`umur`	bilangan bulat 1–120
`Patient Sex`	Orang	`jenis_kelamin`	Male / Female
`Left-Fundus, Right-Fundus`	PemeriksaanMata	`nama_foto`	akhiran _left.jpg / _right.jpg, sesuai ID
`Left- / Right-Diagnostic Keywords`	PemeriksaanMata	`diagnosis`	tidak boleh kosong
`N D G C A H M O`	PasienMata	`kode_diagnosis`	hanya 0/1; N tidak boleh bersama penyakit
`(dikenali dari kata kunci)`	PasienMata	`daftar_penyakit`	harus cocok dengan penanda dokter

Arti 8 kolom penanda: N = Normal, D = Retinopati diabetik, G = Glaukoma, C = Katarak, A = Degenerasi makula, H = Retinopati hipertensi, M = Miopia patologis, O = Kelainan lain. Kolom tidak dipakai: filepath, labels, target, filename (label per foto, turunan dari kolom lain).

Diagram class

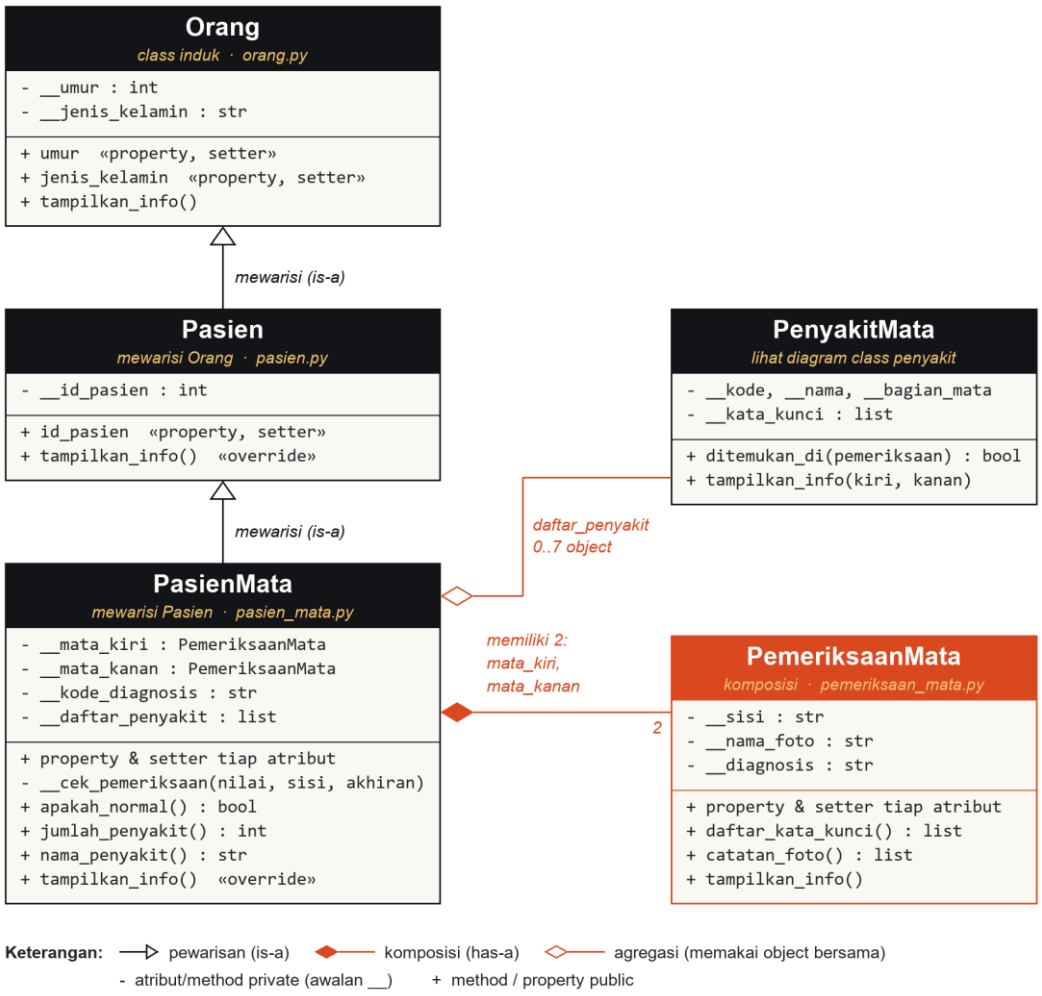
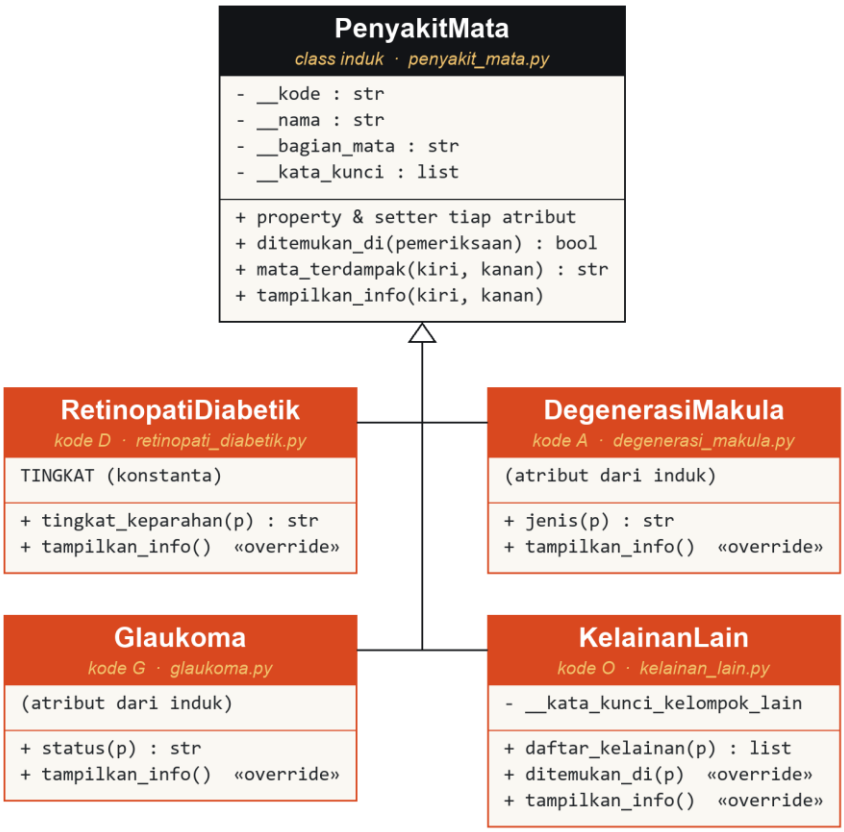


Diagram class 1: Orang → Pasien → PasienMata, komposisi PemeriksaanMata

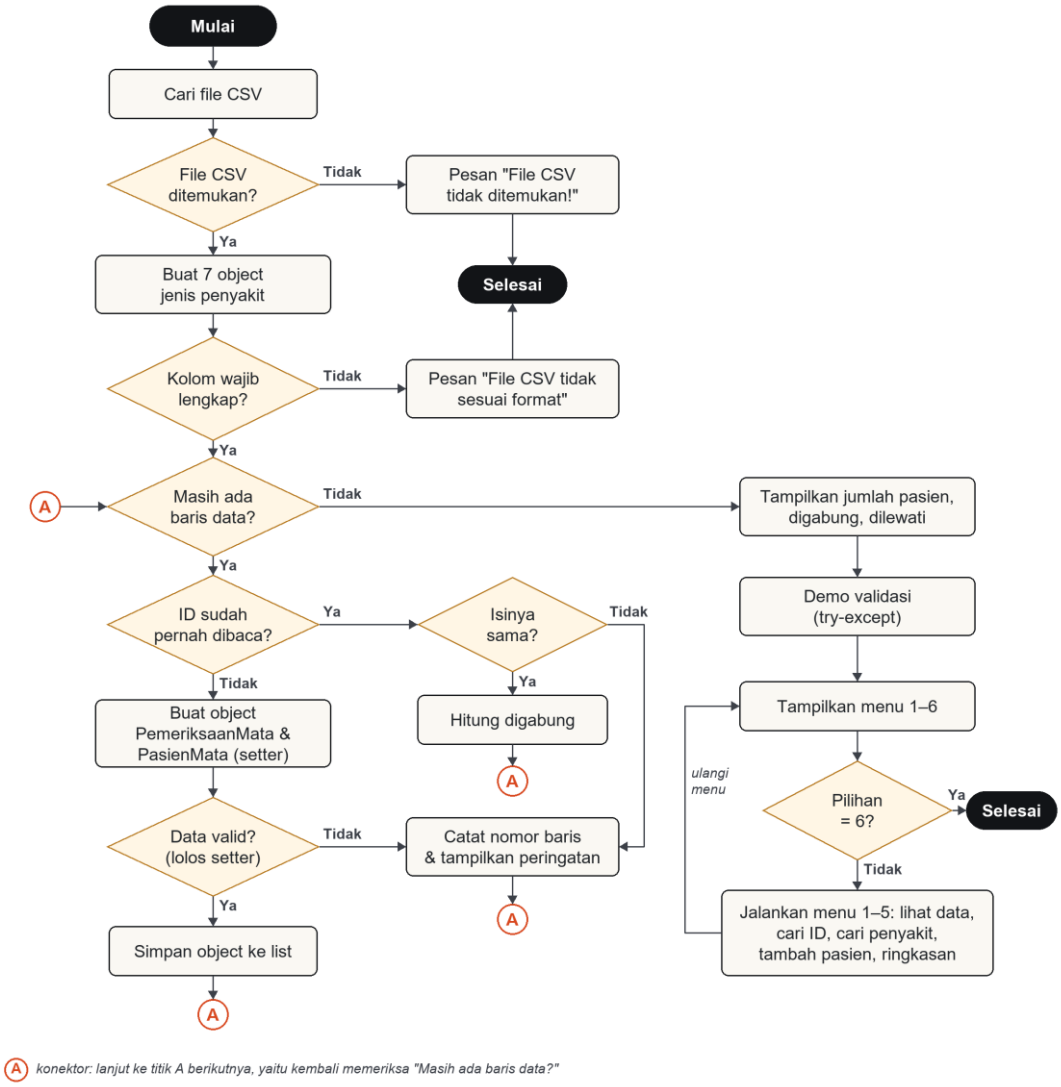


Katarak (C), Retinopati hipertensi (H), dan Miopia patologis (M) dibuat sebagai object langsung dari class PenyakitMata di main.py, karena tidak memerlukan perilaku khusus.

Keterangan: ⟶▷ pewarisan (is-a)
- atribut/method private (awalan __) + method / property public

Diagram class 2: PenyakitMata dan 4 class turunannya

Alur program (flowchart)



Flowchart main.py

Letak konsep OOP di kode (ringkasan untuk presentasi)

Konsep	Letak	Keterangan
Class & Object	`orang.py` baris 6	class induk paling atas (umur, jenis kelamin)
	`pasien.py` baris 8	class Pasien (id_pasien)
	`pasien_mata.py` baris 12	class pasien poli mata
	`pemeriksaan_mata.py` baris 8	class hasil foto satu mata
	`penyakit_mata.py` baris 9	class satu jenis penyakit mata
	`main.py` baris 54	membuat 7 object jenis penyakit
	`main.py` baris 93	satu baris CSV → object PemeriksaanMata & PasienMata
	`main.py` baris 132	object disimpan di list
Enkapsulasi	`orang.py` baris 23	setter umur: bilangan bulat 1–120
	`orang.py` baris 34	setter jenis kelamin: Male/Female
	`pasien.py` baris 22	setter ID: bilangan bulat ≥ 0
	`pemeriksaan_mata.py` baris 40	setter nama foto: akhiran sesuai sisi
	`pasien_mata.py` baris 45	method private: object mata harus sesuai ID
	`pasien_mata.py` baris 58	setter kode diagnosis: N D G C A H M O
	`pasien_mata.py` baris 75	validasi silang penanda vs kata kunci

Konsep	Letak	Keterangan
	`main.py` baris 141	contoh data tidak valid dalam try-except
Inheritance	`pasien.py` baris 13	constructor chaining Pasien → Orang
	`pasien.py` baris 30	override tampilkan_info() milik Orang
	`pasien_mata.py` baris 20	chaining PasienMata → Pasien
	`pasien_mata.py` baris 110	override tampilkan_info() milik Pasien
	`retinopati_diabetik.py` baris 9	turunan PenyakitMata (kode D)
	`degenerasi_makula.py` baris 8	turunan PenyakitMata (kode A)
	`glaukoma.py` baris 8	turunan PenyakitMata (kode G)
	`kelainan_lain.py` baris 48	override ditemukan_di() (polimorfisme)
	`pasien_mata.py` baris 121	polimorfisme: versi method mengikuti class

BAGIAN 1 | RISWAN

Latar Belakang & Struktur Project

Slide 1–5 • **Durasi** ± 3,5 menit • **Konsep** Pengantar • **Fokus** Latar belakang, struktur folder, alur, dan cara menjalankan

Inti yang disampaikan

- Membuka presentasi, memperkenalkan **Kelompok 3**, dan judul project: **Sistem Manajemen Data Pasien Penyakit Mata**.
- Latar belakang: di poli mata, kedua mata pasien difoto dengan kamera fundus (foto bagian dalam mata). Dokter menulis diagnosis berupa **kata kunci** berbahasa Inggris, misalnya *moderate non proliferative retinopathy*.
- Masalahnya: datanya besar (6.392 foto dari 3.358 pasien), satu pasien bisa punya lebih dari satu penyakit (556 pasien), dan data rawan salah input.
- Tujuan: menerapkan **Class & Object**, **Enkapsulasi**, dan **Inheritance** memakai **data nyata** ODIR-5K, bukan data karangan.
- Struktur: folder `program/` berisi **10 file kode**, yaitu 9 file class (satu class satu file, nama snake_case) ditambah `main.py`. Data asli ada di `program/data/full_df.csv`.
- Alur program: baca CSV → setiap baris menjadi object (baris tidak valid dilewati dan dicatat) → demo validasi → menu interaktif 6 pilihan.
- Cara menjalankan: `cd program` lalu `python main.py`. Hanya memakai library bawaan Python (`csv` dan `os`), jadi tidak perlu instalasi tambahan.

Output program yang ditunjukkan

Terminal (slide 5)

```
cd program
python main.py
======================================================================
SISTEM MANAJEMEN DATA PASIEN PENYAKIT MATA
======================================================================
File data          : full_df.csv
Pasien dimuat      : 3358 object PasienMata
Baris digabung     : 3034 (foto mata kedua dari pasien yang sama)
Baris dilewati     : 0
```

> **Kalimat penutup:** “Selanjutnya, Yus Aidil akan menjelaskan dataset yang kami pakai.” Naskah lengkap ada di **Naskah Presentasi Kelompok 3**.

Persiapan tanya jawab

T **Kenapa memakai OOP, bukan program biasa (prosedural)?**

J Karena data pasien punya ciri (atribut) dan perilaku (method) yang jelas. Dengan OOP, setiap pasien menjadi object yang rapi, datanya dijaga lewat enkapsulasi, dan kodenya dipakai ulang lewat inheritance.

T **Apa itu foto fundus?**

J Foto bagian dalam belakang bola mata (retina, makula, dan saraf optik) yang diambil dengan kamera khusus. Banyak penyakit mata, seperti retinopati diabetik dan glaukoma, terlihat di foto ini.

T **Kenapa satu class satu file?**

J Supaya kode mudah dicari, dibaca, dan dibagi tugasnya. Setiap file hanya punya satu tanggung jawab.

T **Apakah harus masuk ke folder program dulu?**

J Disarankan. Tetapi `main.py` mencari file data berdasarkan lokasi file `main.py` sendiri, jadi perintah `python program/main.py` dari folder luar juga berjalan.

BAGIAN 2 | YUS AIDIL

Dataset ODIR-5K & Pemetaan Kolom

Slide 6–9 • **Durasi** ± 3,5 menit • **Konsep** Data nyata • **Fokus** full_df.csv: sumber, kolom, pemetaan ke atribut, dan sebaran penyakit

Inti yang disampaikan

- Sumber data: **ODIR-5K** (Ocular Disease Intelligent Recognition), dikumpulkan Shanggong Medical Technology dari beberapa rumah sakit di Tiongkok dan dipakai pada kompetisi ODIR-2019 Peking University. File `full_df.csv` diunduh dari Kaggle: https://www.kaggle.com/datasets/andrewmvd/ocular-disease-recognition-odir5k.
- Isi: **6.392 baris** (1 baris = 1 foto mata) dan 19 kolom. Setelah digabung per pasien, ada **3.358 pasien**: 1.800 laki-laki dan 1.558 perempuan, umur 1–91 tahun (rata-rata 57,9).
- Kolom yang dipakai (15): `ID`, `Patient Age`, `Patient Sex`, `Left-Fundus`, `Right-Fundus`, `Left-Diagnostic Keywords`, `Right-Diagnostic Keywords`, dan 8 penanda `N D G C A H M O`.
- Arti penanda: **N** normal, **D** retinopati diabetik, **G** glaukoma, **C** katarak, **A** degenerasi makula, **H** retinopati hipertensi, **M** miopia patologis, **O** kelainan lain.
- Kolom `filepath`, `labels`, `target`, dan `filename` tidak dipakai, karena isinya label per foto buatan pengunggah Kaggle (turunan dari kolom lain), bukan data pasien.
- Sebaran: retinopati diabetik 1.105, kelainan lain 905, katarak 208, glaukoma 206, miopia patologis 171, degenerasi makula 163, retinopati hipertensi 103 pasien; 1.080 pasien normal.
- Catatan jujur: ada 15 pasien berumur 1 tahun (kemungkinan salah catat di sumber data). Data tidak kami ubah sedikit pun.

> **Kalimat penutup:** “Selanjutnya, Fahmi akan menjelaskan class dan object.” Naskah lengkap ada di **Naskah Presentasi Kelompok 3**.

Persiapan tanya jawab

T **Kenapa jumlah baris (6.392) berbeda dengan jumlah pasien (3.358)?**

J Karena satu baris mewakili satu foto. Pasien yang kedua fotonya ada muncul di dua baris dengan isi data pasien yang sama. Program menggabungkannya: 3.034 baris digabung.

T **Apakah datanya kalian ubah atau bersihkan dulu?**

J Tidak. File asli dipakai apa adanya; program yang memeriksa setiap nilainya. Hasilnya 0 baris dilewati, artinya data asli lolos semua aturan.

T **Kenapa kolom labels dan target tidak dipakai?**

J Isinya label per foto, bukan per pasien. Contohnya pasien ID 0 berpenanda C (katarak), tetapi baris foto mata kanannya berlabel ['N'] karena mata kanannya normal. Data pasien yang lengkap ada di 8 kolom penanda.

T **Apa yang dimaksud kelainan lain (O)?**

J Kelainan di luar enam kelompok utama. Di data ada 67 jenis, terbanyak macular epiretinal membrane (195 mata) dan drusen (179 mata).

BAGIAN 3 | FAHMI

Class & Object

Slide 10–13 • **Durasi** ± 3,5 menit • **Konsep** Class & Object • **Fokus** Class Orang, constructor, self, dan pembuatan object di buat_pasien()

Inti yang disampaikan

- **Class** adalah cetakan (blueprint) yang berisi atribut (data) dan method (perilaku). **Object** adalah hasil cetakan dengan isi datanya sendiri.
- Project ini punya **9 class**: Orang, Pasien, PasienMata, PemeriksaanMata, PenyakitMata, RetinopatiDiabetik, DegenerasiMakula, Glaukoma, dan KelainanLain.
- **Constructor** `__init__` berjalan otomatis saat object dibuat. `self` adalah object itu sendiri, sehingga setiap object menyimpan datanya masing-masing.
- Object dibuat dengan memanggil nama class seperti fungsi, misalnya `PasienMata(...)`. Di `buat_pasien()`, satu baris CSV menghasilkan 2 object PemeriksaanMata dan 1 object PasienMata.
- Object pasien disimpan di list `daftar_pasien` dengan `append`, sehingga bisa diulang, dicari, dihitung, dan ditambah.
- Saat program berjalan terbentuk **10.081 object**: 3.358 PasienMata + 6.716 PemeriksaanMata + 7 object jenis penyakit.
- Satu class bisa menghasilkan object yang berbeda isi: Katarak, Retinopati hipertensi, dan Miopia patologis adalah 3 object dari class PenyakitMata yang sama.

Kode yang ditunjukkan & fungsi setiap baris

orang.py — baris 6–16 (slide 12)

```
 6  class Orang:
 7      """Menyimpan data dasar seseorang: umur dan jenis kelamin."""
 8
 9      # Nilai jenis kelamin yang ada di data (dicek dari nilai unik kolom Patient Sex)
10      JENIS_KELAMIN_VALID = ("Male", "Female")
11
12      def __init__(self, umur, jenis_kelamin):
13          # [ENKAPSULASI] Nilai diisi lewat setter, jadi langsung divalidasi
14          # sebelum disimpan ke atribut private (self.__umur, self.__jenis_kelamin).
15          self.umur = umur
16          self.jenis_kelamin = jenis_kelamin
```

Baris	Kode	Fungsinya
6	`class Orang:`	Membuat class baru bernama `Orang`, cetakan paling dasar untuk data seseorang.
7	`"""Menyimpan data dasar seseorang: umur dan jenis kelamin."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
9	`# Nilai jenis kelamin yang ada di data (dicek dari nilai unik kolom Patient Sex)`	Komentar penjelas (tidak dijalankan Python).
10	`JENIS_KELAMIN_VALID = ("Male", "Female")`	Konstanta class: jenis kelamin yang sah, diambil dari nilai unik kolom Patient Sex.
12	`def __init__(self, umur, jenis_kelamin):`	**Constructor**: berjalan otomatis saat object dibuat, menerima umur dan jenis kelamin. `self` adalah object itu sendiri.
13	`# [ENKAPSULASI] Nilai diisi lewat setter, jadi langsung divalidasi`	Komentar penjelas (tidak dijalankan Python).
14	`# sebelum disimpan ke atribut private (self.__umur, self.__jenis_kelamin).`	Komentar penjelas (tidak dijalankan Python).

Baris	Kode	Fungsinya
15	`self.umur = umur`	Mengisi `umur` lewat setter-nya, sehingga nilainya langsung diperiksa.
16	`self.jenis_kelamin = jenis_kelamin`	Mengisi `jenis_kelamin` lewat setter-nya, sehingga nilainya langsung diperiksa.

orang.py — baris 42–45 (slide 12)

```
42      def tampilkan_info(self):
43          """Menampilkan data dasar. Method ini di-override oleh class turunan."""
44          print(f"Umur             : {self.umur} tahun")
45          print(f"Jenis Kelamin    : {self.jenis_kelamin}")
```

Baris	Kode	Fungsinya
42	`def tampilkan_info(self):`	Method `tampilkan_info()`: menampilkan data dasar. Nanti ditulis ulang (override) oleh class turunan.
43	`"""Menampilkan data dasar. Method ini di-override oleh class turunan."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
44	`print(f"Umur : {self.umur} tahun")`	Mencetak umur (dibaca lewat getter `self.umur`).
45	`print(f"Jenis Kelamin : {self.jenis_kelamin}")`	Mencetak jenis kelamin.

main.py — baris 93–106 (slide 13)

```
93  def buat_pasien(baris, daftar_jenis):
94      """[CLASS & OBJECT] Mengubah satu baris CSV menjadi object PasienMata."""
95      # [KOMPOSISI] Object PemeriksaanMata dibuat dulu, lalu dimasukkan ke PasienMata
96      mata_kiri = PemeriksaanMata("kiri", baris["Left-Fundus"].strip(), baris["Left-Diagnostic
    Keywords"])
97      mata_kanan = PemeriksaanMata("kanan", baris["Right-Fundus"].strip(), baris["Right-
    Diagnostic Keywords"])
98      return PasienMata(
99          ke_bilangan_bulat(baris["ID"], "ID"),
100         ke_bilangan_bulat(baris["Patient Age"], "Umur"),
101         baris["Patient Sex"].strip(),
102         mata_kiri,
103         mata_kanan,
104         baca_kode_diagnosis(baris),
105         deteksi_penyakit(mata_kiri, mata_kanan, daftar_jenis),
106     )
```

Baris	Kode	Fungsinya
93	`def buat_pasien(baris, daftar_jenis):`	Fungsi untuk mengubah satu baris CSV (dictionary) menjadi object `PasienMata`.
94	`"""[CLASS & OBJECT] Mengubah satu baris CSV menjadi object PasienMata."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
95	`# [KOMPOSISI] Object PemeriksaanMata dibuat dulu, lalu dimasukkan ke PasienMata`	Komentar penjelas (tidak dijalankan Python).
96	`mata_kiri = PemeriksaanMata("kiri", baris["Left-Fundus"].strip(), baris["Left-Diagnostic Keywords"])`	Membuat object `PemeriksaanMata` mata kiri dari kolom Left-Fundus dan Left-Diagnostic Keywords.
97	`mata_kanan = PemeriksaanMata("kanan", baris["Right-Fundus"].strip(), baris["Right-Diagnostic Keywords"])`	Membuat object `PemeriksaanMata` mata kanan dari kolom Right-Fundus dan Right-Diagnostic Keywords.
98	`return PasienMata(`	Membuat object `PasienMata` dan langsung mengembalikannya, dengan isian:
99	`ke_bilangan_bulat(baris["ID"], "ID"),`	ID dari kolom ID, diubah menjadi bilangan bulat,
100	`ke_bilangan_bulat(baris["Patient Age"], "Umur"),`	umur dari kolom Patient Age,
101	`baris["Patient Sex"].strip(),`	jenis kelamin dari kolom Patient Sex,
102	`mata_kiri,`	object mata kiri,

Baris	Kode	Fungsinya
103	`mata_kanan,`	object mata kanan,
104	`baca_kode_diagnosis(baris),`	kode diagnosis dari 8 kolom penanda,
105	`deteksi_penyakit(mata_kiri, mata_kanan, daftar_jenis),`	dan daftar penyakit hasil deteksi kata kunci.
106	`)`	Penutup tanda kurung yang dibuka di atasnya.

main.py — baris 132–132 (slide 13, kode terkait (dibahas, tidak tampil di slide))

```
132                 daftar_pasien.append(buat_pasien(baris, daftar_jenis))
```

Baris	Kode	Fungsinya
132	`daftar_pasien.append(buat_pasien(baris, daftar_jenis))`	Pasien baru: ubah baris menjadi object lewat `buat_pasien()`, lalu simpan ke list dengan `append`.

> **Kalimat penutup:** "Selanjutnya, Diva akan menjelaskan bagaimana data di dalam object dijaga." Naskah lengkap ada di **Naskah Presentasi Kelompok 3**.

Persiapan tanya jawab

T **Apa beda class dan object?**

J Class adalah rancangannya, object adalah wujudnya. Class PasienMata hanya ditulis sekali, tetapi object-nya ada 3.358, satu untuk setiap pasien.

T **Apa fungsi self?**

J `self` menunjuk ke object yang sedang dipakai. Dengan `self`, setiap object menyimpan dan membaca datanya sendiri, misalnya umur pasien ID 0 tidak tercampur dengan pasien ID 1.

T **Kapan __init__ dijalankan?**

J Otomatis setiap kali object dibuat, misalnya saat `PasienMata(...)` dipanggil di `buat_pasien()`.

T **Kenapa object disimpan di list?**

J Supaya semua pasien bisa diproses bersama: diulang dengan `for`, dicari berdasarkan ID, dihitung ringkasannya, dan ditambah pasien baru dengan `append`.

BAGIAN 4 | DIVA NURISLAMIA AGFA

Enkapsulasi

Slide 14–18 • **Durasi** ± 4 menit • **Konsep** Enkapsulasi • **Fokus** Atribut private, getter & setter, validasi silang, dan demo data tidak valid

Inti yang disampaikan

- Semua atribut dibuat **private** dengan awalan dua garis bawah, misalnya `__umur`. Python mengubah namanya menjadi `_Orang__umur` (name mangling), sehingga `pasien.__umur` menghasilkan AttributeError.
- **Getter** memakai `@property` untuk membaca nilai; **setter** memakai `@nama.setter` untuk mengubah nilai setelah divalidasi. Nilai salah ditolak dengan `raise ValueError`.
- Constructor mengisi atribut lewat setter (`self.umur = umur`), jadi data dari CSV juga divalidasi saat object dibuat.
- Total ada **15 setter** di project ini. Aturan utama data pasien: umur 1–120, jenis kelamin Male/Female, ID ≥ 0, sisi kiri/kanan, nama foto sesuai sisi dan ID, diagnosis tidak kosong, kode diagnosis valid.
- **Validasi silang** di setter `daftar_penyakit`: penyakit yang dikenali dari kata kunci harus sama dengan penanda dokter. Hasil pengujian: 3.358 pasien cocok 100%.
- Getter `daftar_penyakit` mengembalikan **salinan list**, sehingga isi asli di dalam object tidak bisa diubah dari luar.
- Demo di `demo_validasi()`: `pasien.umur = -5` ditolak dan umur tetap 69; pasien berpenanda katarak tetapi diagnosisnya normal juga ditolak.

Kode yang ditunjukkan & fungsi setiap baris

orang.py — baris 19–28 (slide 16)

```
19      @property
20      def umur(self):
21          return self.__umur
22
23      @umur.setter
24      def umur(self, nilai):
25          # Aturan: umur bilangan bulat 1 sampai 120 tahun
26          if not isinstance(nilai, int) or nilai < 1 or nilai > 120:
27              raise ValueError(f"Umur harus bilangan bulat 1 sampai 120, bukan {nilai}")
28          self.__umur = nilai
```

Baris	Kode	Fungsinya
19	`@property`	Dekorator `@property`: method di bawahnya menjadi **getter**, sehingga nilainya dibaca seperti atribut.
20	`def umur(self):`	Getter `umur`: dijalankan saat kode membaca `objek.umur`.
21	`return self.__umur`	Mengembalikan nilai atribut private `__umur`.
23	`@umur.setter`	Dekorator setter: method di bawahnya dijalankan setiap kali `umur` diubah.
24	`def umur(self, nilai):`	Setter `umur`: menerima nilai baru yang diperiksa sebelum disimpan.
25	`# Aturan: umur bilangan bulat 1 sampai 120 tahun`	Komentar penjelas (tidak dijalankan Python).
26	`if not isinstance(nilai, int) or nilai < 1 or nilai > 120:`	Validasi: menolak nilai yang bukan bilangan bulat (`int`) atau di luar 1 sampai 120.
27	`raise ValueError(f"Umur harus bilangan bulat 1 sampai 120, bukan {nilai}")`	Memunculkan `ValueError` beserta pesan kesalahan, sehingga nilai yang salah tidak disimpan.
28	`self.__umur = nilai`	Nilai lolos pemeriksaan, lalu disimpan ke atribut private `__umur`.

pasien_mata.py — baris 71–73 (slide 16)

```
71      @property
72      def daftar_penyakit(self):
73          return list(self.__daftar_penyakit)  # kirim salinan agar isi asli tidak diubah dari
    luar
```

Baris	Kode	Fungsinya
71	`@property`	Dekorator `@property`: method di bawahnya menjadi **getter**, sehingga nilainya dibaca seperti atribut.
72	`def daftar_penyakit(self):`	Getter `daftar_penyakit`: dijalankan saat kode membaca `objek.daftar_penyakit`.
73	`return list(self.__daftar_penyakit)  # kirim salinan agar isi asli tidak diubah dari luar`	Getter mengirim **salinan list**, sehingga list asli di dalam object tidak bisa diubah dari luar.

pasien_mata.py — baris 75–90 (slide 17)

```
75      @daftar_penyakit.setter
76      def daftar_penyakit(self, nilai):
77          if not isinstance(nilai, list):
78              raise ValueError("Daftar penyakit harus berupa list")
79          kode_ditemukan = ""
80          for penyakit in nilai:
81              if not isinstance(penyakit, PenyakitMata):
82                  raise ValueError("Isi daftar penyakit harus object PenyakitMata")
83              kode_ditemukan += penyakit.kode
84          # Validasi silang: penyakit yang dikenali dari kata kunci diagnosis harus sama
85          # dengan penanda yang diberi dokter (kolom D, G, C, A, H, M, O).
86          kode_penanda = self.kode_diagnosis.replace("N", "")
87          if sorted(kode_ditemukan) != sorted(kode_penanda):
88              raise ValueError(f"Penanda diagnosis '{self.kode_diagnosis}' tidak cocok dengan "
89                               f"kata kunci diagnosis (terbaca: '{kode_ditemukan or 'N'}')")
90          self.__daftar_penyakit = list(nilai)
```

Baris	Kode	Fungsinya
75	`@daftar_penyakit.setter`	Dekorator setter: method di bawahnya dijalankan setiap kali `daftar_penyakit` diubah.
76	`def daftar_penyakit(self, nilai):`	Setter `daftar_penyakit`: menerima nilai baru yang diperiksa sebelum disimpan.
77	`if not isinstance(nilai, list):`	Validasi: daftar penyakit harus berupa list.
78	`raise ValueError("Daftar penyakit harus berupa list")`	Memunculkan `ValueError` beserta pesan kesalahan, sehingga nilai yang salah tidak disimpan.
79	`kode_ditemukan = ""`	Menyiapkan teks kosong untuk mengumpulkan kode penyakit yang dikenali.
80	`for penyakit in nilai:`	Memeriksa setiap isi list.
81	`if not isinstance(penyakit, PenyakitMata):`	Setiap isi harus object `PenyakitMata` (atau turunannya).
82	`raise ValueError("Isi daftar penyakit harus object PenyakitMata")`	Memunculkan `ValueError` beserta pesan kesalahan, sehingga nilai yang salah tidak disimpan.
83	`kode_ditemukan += penyakit.kode`	Mengumpulkan kode penyakit, misalnya 'D' dan 'O' menjadi 'DO'.
84	`# Validasi silang: penyakit yang dikenali dari kata kunci diagnosis harus sama`	Komentar penjelas (tidak dijalankan Python).
85	`# dengan penanda yang diberi dokter (kolom D, G, C, A, H, M, O).`	Komentar penjelas (tidak dijalankan Python).
86	`kode_penanda = self.kode_diagnosis.replace("N", "")`	Mengambil kode penanda dokter tanpa huruf N.
87	`if sorted(kode_ditemukan) != sorted(kode_penanda):`	**Validasi silang**: membandingkan kedua kode tanpa memedulikan urutan huruf.

Baris	Kode	Fungsinya
88	`raise ValueError(f"Penanda diagnosis '{self.kode_diagnosis}' tidak cocok dengan "`	Memunculkan `ValueError` beserta pesan kesalahan, sehingga nilai yang salah tidak disimpan.
89	`f"kata kunci diagnosis (terbaca: '{kode_ditemukan or 'N'}')")`	Lanjutan pesan error: menampilkan kode yang terbaca dari kata kunci ('N' jika tidak ada).
90	`self.__daftar_penyakit = list(nilai)`	Menyimpan salinan list ke atribut private.

main.py — baris 144–151 (slide 18)

```
144     pasien = daftar_pasien[0]
145     print(f"Umur pasien ID {pasien.id_pasien} sebelum diubah : {pasien.umur}")
146     print("Mencoba mengubah: pasien.umur = -5")
147     try:
148         pasien.umur = -5  # tidak valid, umur harus 1 sampai 120
149     except ValueError as error:
150         print(f"ValueError tertangkap: {error}")
151     print(f"Umur pasien ID {pasien.id_pasien} sesudahnya     : {pasien.umur} (tidak berubah)")
```

Baris	Kode	Fungsinya
144	`pasien = daftar_pasien[0]`	Mengambil object pasien pertama (ID 0) sebagai contoh.
145	`print(f"Umur pasien ID {pasien.id_pasien} sebelum diubah : {pasien.umur}")`	Mencetak umur sebelum dicoba diubah.
146	`print("Mencoba mengubah: pasien.umur = -5")`	Memberi tahu nilai yang akan dicoba.
147	`try:`	`try`: menjalankan kode di bawahnya sambil berjaga-jaga jika muncul error.
148	`pasien.umur = -5  # tidak valid, umur harus 1 sampai 120`	Mengisi umur dengan -5 lewat setter; setter memunculkan `ValueError`.
149	`except ValueError as error:`	Menangkap `ValueError` dari blok `try`; pesannya disimpan di `error`, dan program tidak berhenti.
150	`print(f"ValueError tertangkap: {error}")`	Mencetak pesan error dari setter.
151	`print(f"Umur pasien ID {pasien.id_pasien} sesudahnya     : {pasien.umur} (tidak berubah)")`	Membuktikan umur tidak berubah, karena nilai salah tidak pernah disimpan.

main.py — baris 153–159 (slide 18, kode terkait (dibahas, tidak tampil di slide))

```
153     print("Mencoba membuat pasien berpenanda katarak (C), tetapi diagnosisnya normal")
154     try:
155         mata_kiri = PemeriksaanMata("kiri", "9999_left.jpg", "normal fundus")
156         mata_kanan = PemeriksaanMata("kanan", "9999_right.jpg", "normal fundus")
157         PasienMata(9999, 50, "Male", mata_kiri, mata_kanan, "C", [])
158     except ValueError as error:
159         print(f"ValueError tertangkap: {error}")
```

Baris	Kode	Fungsinya
153	`print("Mencoba membuat pasien berpenanda katarak (C), tetapi diagnosisnya normal")`	Memberi tahu contoh kedua: validasi silang.
154	`try:`	`try`: menjalankan kode di bawahnya sambil berjaga-jaga jika muncul error.
155	`mata_kiri = PemeriksaanMata("kiri", "9999_left.jpg", "normal fundus")`	Membuat object mata kiri dengan diagnosis normal.
156	`mata_kanan = PemeriksaanMata("kanan", "9999_right.jpg", "normal fundus")`	Membuat object mata kanan dengan diagnosis normal.
157	`PasienMata(9999, 50, "Male", mata_kiri, mata_kanan, "C", [])`	Membuat pasien berpenanda C (katarak) tanpa penyakit terdeteksi: ditolak setter `daftar_penyakit`.
158	`except ValueError as error:`	Menangkap `ValueError` dari blok `try`; pesannya disimpan di `error`, dan program tidak berhenti.
159	`print(f"ValueError tertangkap: {error}")`	Mencetak pesan error dari setter.

Output program yang ditunjukkan

python main.py (slide 18)

```
Umur pasien ID 0 sebelum diubah : 69
Mencoba mengubah: pasien.umur = -5
ValueError tertangkap: Umur harus bilangan bulat 1 sampai 120, bukan -5
Umur pasien ID 0 sesudahnya     : 69 (tidak berubah)

Mencoba membuat pasien berpenanda katarak (C), tetapi diagnosisnya normal
ValueError tertangkap: Penanda diagnosis 'C' tidak cocok dengan kata kunci diagnosis (terbaca: 'N')
```

> **Kalimat penutup:** “Selanjutnya, Rosa akan menjelaskan konsep inheritance.” Naskah lengkap ada di **Naskah Presentasi Kelompok 3**.

Persiapan tanya jawab

T **Kenapa memakai dua garis bawah, bukan satu?**

J Satu garis bawah hanya kesepakatan bahwa atribut itu internal. Dua garis bawah membuat Python mengganti namanya (name mangling), sehingga tidak bisa diakses langsung dengan nama aslinya.

T **Apakah atribut private benar-benar tidak bisa diakses?**

J Secara teknis masih bisa lewat nama `_Orang__umur`, tetapi itu melanggar aturan. Tujuan enkapsulasi adalah mencegah perubahan yang tidak sengaja dan memastikan semua perubahan lewat setter.

T **Kenapa constructor menulis self.umur, bukan self.__umur?**

J Supaya nilai awal juga melewati setter dan ikut divalidasi. Kalau langsung ke `self.__umur`, data salah dari CSV bisa lolos.

T **Kenapa memakai ValueError?**

J ValueError adalah exception bawaan Python untuk nilai yang tidak sesuai aturan. Pemanggil bisa menangkapnya dengan try-except, sehingga program tidak berhenti.

BAGIAN 5 | ROSA RAMADANI

Inheritance: Pewarisan Bertingkat

Slide 19–22 • **Durasi** ± 3 menit • **Konsep** Inheritance • **Fokus** Orang → Pasien → PasienMata, super(), dan method overriding

Inti yang disampaikan

- **Inheritance**: class anak mewarisi atribut dan method class induk, ditulis dengan nama induk di dalam kurung, misalnya `class Pasien(Orang)`.
- Pewarisan **bertingkat** tiga tingkat: Orang (umur, jenis kelamin) → Pasien (+ id_pasien) → PasienMata (+ mata kiri, mata kanan, kode diagnosis, daftar penyakit).
- `super().__init__(...)` menjalankan constructor induk (**constructor chaining**), sehingga validasi umur dan jenis kelamin tetap diurus class Orang.
- **Method overriding**: `tampilkan_info()` ditulis ulang di ketiga tingkat, dan setiap versi memanggil versi induknya dengan `super().tampilkan_info()`.
- Bukti di output pasien ID 188: baris ID dari Pasien, umur dan jenis kelamin dari Orang, data mata dan penyakit dari PasienMata.
- Urutan pencarian method (MRO): PasienMata → Pasien → Orang → object, dan `isinstance(pasien, Orang)` bernilai True.
- Ada keluarga pewarisan kedua: PenyakitMata dengan 4 class turunan (dijelaskan Revalinda).

Kode yang ditunjukkan & fungsi setiap baris

pasien.py — baris 8–15 (slide 21)

```
 8  class Pasien(Orang):
 9      """Pasien rumah sakit (turunan dari Orang)."""
10
11      def __init__(self, id_pasien, umur, jenis_kelamin):
12          # [INHERITANCE] Constructor chaining: umur & jenis kelamin diurus oleh Orang
13          super().__init__(umur, jenis_kelamin)
14          # [ENKAPSULASI] Diisi lewat setter agar tervalidasi
15          self.id_pasien = id_pasien
```

Baris	Kode	Fungsinya
8	`class Pasien(Orang):`	Membuat class `Pasien` yang **mewarisi** `Orang`; nama class induk ditulis di dalam kurung.
9	`"""Pasien rumah sakit (turunan dari Orang)."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
11	`def __init__(self, id_pasien, umur, jenis_kelamin):`	Constructor Pasien: menerima ID, umur, dan jenis kelamin.
12	`# [INHERITANCE] Constructor chaining: umur & jenis kelamin diurus oleh Orang`	Komentar penjelas (tidak dijalankan Python).
13	`super().__init__(umur, jenis_kelamin)`	**Constructor chaining**: menjalankan constructor `Orang` untuk mengisi dan memvalidasi umur serta jenis kelamin.
14	`# [ENKAPSULASI] Diisi lewat setter agar tervalidasi`	Komentar penjelas (tidak dijalankan Python).
15	`self.id_pasien = id_pasien`	Mengisi `id_pasien` lewat setter-nya, sehingga nilainya langsung diperiksa.

pasien.py — baris 29–33 (slide 21)

```
29      # ---------- [INHERITANCE] Method overriding ----------
30      def tampilkan_info(self):
31          """Override tampilkan_info() milik Orang: tambah ID pasien."""
32          print(f"ID Pasien        : {self.id_pasien}")
33          super().tampilkan_info()  # memakai ulang tampilan umur & jenis kelamin dari Orang
```

Baris	Kode	Fungsinya
29	`# ---------- [INHERITANCE] Method overriding ----------`	Komentar pembatas bagian kode (tidak dijalankan Python).
30	`def tampilkan_info(self):`	**Method overriding**: menulis ulang `tampilkan_info()` milik `Orang`.
31	`"""Override tampilkan_info() milik Orang: tambah ID pasien."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
32	`print(f"ID Pasien : {self.id_pasien}")`	Mencetak ID pasien.
33	`super().tampilkan_info()  # memakai ulang tampilan umur & jenis kelamin dari Orang`	Memanggil `tampilkan_info()` milik `Orang` lewat `super()`, sehingga umur dan jenis kelamin ikut tercetak.

pasien_mata.py — baris 12–12 (slide 22)

```
12  class PasienMata(Pasien):
```

Baris	Kode	Fungsinya
12	`class PasienMata(Pasien):`	Membuat class `PasienMata` yang mewarisi `Pasien` (tingkat ketiga: Orang → Pasien → PasienMata).

pasien_mata.py — baris 17–20 (slide 22)

```
17      def __init__(self, id_pasien, umur, jenis_kelamin, mata_kiri, mata_kanan,
18                   kode_diagnosis, daftar_penyakit):
19          # [INHERITANCE] Constructor chaining: Pasien -> Orang
20          super().__init__(id_pasien, umur, jenis_kelamin)
```

Baris	Kode	Fungsinya
17	`def __init__(self, id_pasien, umur, jenis_kelamin, mata_kiri, mata_kanan,`	Constructor PasienMata menerima tujuh data …
18	`kode_diagnosis, daftar_penyakit):`	… termasuk kode diagnosis dan daftar penyakit.
19	`# [INHERITANCE] Constructor chaining: Pasien -> Orang`	Komentar penjelas (tidak dijalankan Python).
20	`super().__init__(id_pasien, umur, jenis_kelamin)`	**Constructor chaining** ke `Pasien`, yang lalu meneruskan umur dan jenis kelamin ke `Orang`.

pasien_mata.py — baris 110–114 (slide 22)

```
110     def tampilkan_info(self):
111         """Override tampilkan_info() milik Pasien: tambah hasil pemeriksaan dan penyakit."""
112         super().tampilkan_info()  # tampilkan ID, umur, jenis kelamin
113         self.mata_kiri.tampilkan_info()
114         self.mata_kanan.tampilkan_info()
```

Baris	Kode	Fungsinya
110	`def tampilkan_info(self):`	**Method overriding**: menulis ulang `tampilkan_info()` milik `Pasien`.
111	`"""Override tampilkan_info() milik Pasien: tambah hasil pemeriksaan dan penyakit."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
112	`super().tampilkan_info()  # tampilkan ID, umur, jenis kelamin`	Memanggil versi `Pasien` (yang juga memanggil versi `Orang`): ID, umur, dan jenis kelamin.
113	`self.mata_kiri.tampilkan_info()`	Komposisi: object mata kiri menampilkan diagnosisnya sendiri.
114	`self.mata_kanan.tampilkan_info()`	Object mata kanan menampilkan diagnosisnya.

Output program yang ditunjukkan

Menu 2 · cari pasien ID 188 (slide 22)

```
Masukkan ID pasien: 188
ID Pasien        : 188
Umur             : 55 tahun
Jenis Kelamin    : Female
Mata kiri        : laser spot, severe proliferative diabetic retinopathy
Mata kanan       : cataract
Penyakit         : 3 kelompok
```

> **Kalimat penutup:** “Selanjutnya, Sofiang akan menjelaskan konsep komposisi.” Naskah lengkap ada di **Naskah Presentasi Kelompok 3**.

Persiapan tanya jawab

T **Apa beda inheritance dan komposisi?**

J Inheritance adalah hubungan 'adalah' (PasienMata adalah Pasien). Komposisi adalah hubungan 'memiliki' (PasienMata memiliki dua PemeriksaanMata).

T **Apa yang terjadi kalau super().__init__ tidak dipanggil?**

J Atribut umur dan jenis kelamin tidak pernah diisi, sehingga saat dibaca akan muncul AttributeError.

T **Apa itu method overriding?**

J Menulis ulang method milik induk dengan nama yang sama di class anak, supaya perilakunya disesuaikan. Contohnya `tampilkan_info()` di PasienMata menambah data mata dan penyakit.

T **Kenapa harus bertingkat, tidak langsung PasienMata(Orang)?**

J Pasien berlaku umum untuk semua pasien (punya ID), sedangkan PasienMata khusus poli mata. Kalau nanti ada poli lain, cukup membuat turunan baru dari Pasien.

BAGIAN 6 | SOFIANG

Komposisi: Pemeriksaan Mata

Slide 23–26 • **Durasi** ± 3 menit • **Konsep** Komposisi • **Fokus** Class PemeriksaanMata, hubungan 'memiliki', dan validasi object

Inti yang disampaikan

- **Komposisi** adalah hubungan 'memiliki' (has-a): satu PasienMata memiliki dua object PemeriksaanMata (kiri dan kanan) dan list object PenyakitMata.
- PemeriksaanMata tidak mewarisi class apa pun, karena mata bukan pasien; mata adalah bagian dari data pasien.
- Atribut PemeriksaanMata: `sisi` (kiri/kanan), `nama_foto` (berakhiran `_left.jpg`/`_right.jpg`), dan `diagnosis` (tidak boleh kosong). Semuanya private dengan setter.
- Method private `__cek_pemeriksaan()` di PasienMata memakai `isinstance` dan memastikan nama foto sesuai ID pasien, sehingga data mata pasien lain tidak bisa tertukar.
- `daftar_kata_kunci()` mengganti **koma lebar** (,) menjadi koma biasa; koma lebar dipakai pada data 567 pasien.
- `catatan_foto()` memisahkan catatan kualitas foto yang bukan penyakit, misalnya *lens dust* (debu lensa kamera) di 225 foto.
- Total ada **6.716 object PemeriksaanMata**, dua untuk setiap pasien.

Kode yang ditunjukkan & fungsi setiap baris

pemeriksaan_mata.py — baris 40–46 (slide 25)

```
40      @nama_foto.setter
41      def nama_foto(self, nilai):
42          # Aturan data: foto mata kiri berakhiran _left.jpg, mata kanan _right.jpg
43          akhiran = "_left.jpg" if self.sisi == "kiri" else "_right.jpg"
44          if not isinstance(nilai, str) or not nilai.endswith(akhiran):
45              raise ValueError(f"Nama foto mata {self.sisi} harus berakhiran {akhiran}, bukan
    '{nilai}'")
46          self.__nama_foto = nilai
```

Baris	Kode	Fungsinya
40	`@nama_foto.setter`	Dekorator setter: method di bawahnya dijalankan setiap kali `nama_foto` diubah.
41	`def nama_foto(self, nilai):`	Setter `nama_foto`: menerima nilai baru yang diperiksa sebelum disimpan.
42	`# Aturan data: foto mata kiri berakhiran _left.jpg, mata kanan _right.jpg`	Komentar penjelas (tidak dijalankan Python).
43	`akhiran = "_left.jpg" if self.sisi == "kiri" else "_right.jpg"`	Menentukan akhiran nama foto yang benar berdasarkan sisi mata.
44	`if not isinstance(nilai, str) or not nilai.endswith(akhiran):`	Validasi: nama foto harus teks yang berakhiran sesuai sisi.
45	`raise ValueError(f"Nama foto mata {self.sisi} harus berakhiran {akhiran}, bukan '{nilai}'")`	Memunculkan `ValueError` beserta pesan kesalahan, sehingga nilai yang salah tidak disimpan.
46	`self.__nama_foto = nilai`	Nilai lolos pemeriksaan, lalu disimpan ke atribut private `__nama_foto`.

pasien_mata.py — baris 45–52 (slide 25)

```
45      def __cek_pemeriksaan(self, nilai, sisi, akhiran):
46          """Method private: pemeriksaan harus object PemeriksaanMata, sisinya benar,
47          dan nama fotonya sesuai ID pasien (contoh: ID 7 -> 7_left.jpg)."""
48          if not isinstance(nilai, PemeriksaanMata) or nilai.sisi != sisi:
49              raise ValueError(f"Mata {sisi} harus berupa object PemeriksaanMata sisi {sisi}")
50          if nilai.nama_foto != f"{self.id_pasien}{akhiran}":
51              raise ValueError(f"Nama foto '{nilai.nama_foto}' tidak sesuai dengan ID pasien
    {self.id_pasien}")
52          return nilai
```

Baris	Kode	Fungsinya
45	`def __cek_pemeriksaan(self, nilai, sisi, akhiran):`	**Method private** (awalan __): hanya dipakai di dalam class untuk memeriksa object mata.
46	`"""Method private: pemeriksaan harus object PemeriksaanMata, sisinya benar,`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
47	`dan nama fotonya sesuai ID pasien (contoh: ID 7 -> 7_left.jpg)."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
48	`if not isinstance(nilai, PemeriksaanMata) or nilai.sisi != sisi:`	Validasi: harus object `PemeriksaanMata` (dicek dengan `isinstance`) dan sisinya benar.
49	`raise ValueError(f"Mata {sisi} harus berupa object PemeriksaanMata sisi {sisi}")`	Memunculkan `ValueError` beserta pesan kesalahan, sehingga nilai yang salah tidak disimpan.
50	`if nilai.nama_foto != f"{self.id_pasien}{akhiran}":`	Validasi: nama foto harus sesuai ID pasien, misalnya ID 7 → `7_left.jpg`.
51	`raise ValueError(f"Nama foto '{nilai.nama_foto}' tidak sesuai dengan ID pasien {self.id_pasien}")`	Memunculkan `ValueError` beserta pesan kesalahan, sehingga nilai yang salah tidak disimpan.
52	`return nilai`	Mengembalikan object yang sudah lolos pemeriksaan.

pemeriksaan_mata.py — baris 16–16 (slide 26)

```
16      KOMA_LEBAR = "\uff0c"  # tanda koma lebar (，) yang dipakai di data sebagai pemisah
```

Baris	Kode	Fungsinya
16	`KOMA_LEBAR = "\uff0c"  # tanda koma lebar (，) yang dipakai di data sebagai pemisah`	Konstanta koma lebar (，) berkode Unicode `\uff0c`, yang dipakai sebagai pemisah diagnosis pada sebagian data.

pemeriksaan_mata.py — baris 60–75 (slide 26)

```
60      def daftar_kata_kunci(self):
61          """Memecah diagnosis menjadi daftar kata kunci berhuruf kecil."""
62          teks = self.diagnosis.replace(PemeriksaanMata.KOMA_LEBAR, ",").lower()
63          hasil = []
64          for kata in teks.split(","):
65              if kata.strip() != "":
66                  hasil.append(kata.strip())
67          return hasil
68
69      def catatan_foto(self):
70          """Catatan kualitas foto (bukan penyakit), misalnya 'lens dust'."""
71          hasil = []
72          for kata in self.daftar_kata_kunci():
73              if kata in PemeriksaanMata.BUKAN_PENYAKIT and kata != "normal fundus":
74                  hasil.append(kata)
75          return hasil
```

Baris	Kode	Fungsinya
60	`def daftar_kata_kunci(self):`	Method untuk memecah diagnosis menjadi list kata kunci.
61	`"""Memecah diagnosis menjadi daftar kata kunci berhuruf kecil."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
62	`teks = self.diagnosis.replace(PemeriksaanMata.KOMA_LEBAR, ",").lower()`	Mengganti koma lebar menjadi koma biasa, lalu mengubah semua huruf menjadi huruf kecil.

Baris	Kode	Fungsinya
63	`hasil = []`	Menyiapkan list kosong `hasil`.
64	`for kata in teks.split(","):`	Memecah teks di setiap tanda koma, lalu mengulang setiap potongannya.
65	`if kata.strip() != "":`	Potongan yang kosong diabaikan.
66	`hasil.append(kata.strip())`	Menambahkan kata kunci (tanpa spasi di tepi) ke list `hasil`.
67	`return hasil`	Mengembalikan list kata kunci.
69	`def catatan_foto(self):`	Method untuk mengambil catatan kualitas foto yang bukan penyakit.
70	`"""Catatan kualitas foto (bukan penyakit), misalnya 'lens dust'."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
71	`hasil = []`	Menyiapkan list kosong `hasil`.
72	`for kata in self.daftar_kata_kunci():`	Mengulang setiap kata kunci diagnosis mata ini.
73	`if kata in PemeriksaanMata.BUKAN_PENYAKIT and kata != "normal fundus":`	Memilih kata yang termasuk `BUKAN_PENYAKIT`, kecuali 'normal fundus' (hasil normal, bukan catatan foto).
74	`hasil.append(kata)`	Menyimpan catatan foto ke list `hasil`.
75	`return hasil`	Mengembalikan list catatan foto.

Kalimat penutup: “Selanjutnya, Revalinda akan menjelaskan class-class penyakit mata.” Naskah lengkap ada di **Naskah Presentasi Kelompok 3**.

Persiapan tanya jawab

T **Kenapa PemeriksaanMata tidak mewarisi Pasien?**

J Karena hubungannya bukan 'adalah'. Mata bukan pasien, tetapi milik pasien, jadi yang tepat adalah komposisi.

T **Apa itu koma lebar dan kenapa perlu ditangani?**

J Koma lebar (，) adalah tanda koma dari huruf Tiongkok. Di data asli, sebagian diagnosis memakai koma ini sebagai pemisah. Tanpa penanganan, dua diagnosis akan terbaca sebagai satu.

T **Apa itu lens dust?**

J Catatan bahwa ada debu di lensa kamera saat memotret. Ini catatan kualitas foto, bukan penyakit, jadi tidak dihitung sebagai penyakit.

T **Kenapa sisi diisi sebelum nama_foto di constructor?**

J Karena setter `nama_foto` memakai `self.sisi` untuk menentukan akhiran yang benar (_left.jpg atau _right.jpg).

BAGIAN 7 | REVALINDA

Hierarki Class Penyakit & Polimorfisme

Slide 27–30 • **Durasi** ± 3,5 menit • **Konsep** Inheritance & Polimorfisme • **Fokus** PenyakitMata dan 4 class turunan, method khusus, overriding, polimorfisme

Inti yang disampaikan

- Class induk **PenyakitMata**: kode (D, G, C, A, H, M, O), nama, bagian_mata, dan kata_kunci, dengan method `ditemukan_di()`, `mata_terdampak()`, dan `tampilkan_info()`.
- **RetinopatiDiabetik** menambah `tingkat_keparahan()`: dari 1.764 mata, sedang 977, ringan 543, berat 158, proliferatif 32, tidak disebutkan 54.
- **DegenerasiMakula** menambah `jenis()`: kering 227, basah 49 mata. **Glaukoma** menambah `status()`: pasti 274, dugaan 42 mata.
- **KelainanLain** menambah `daftar_kelainan()` dan meng-override `ditemukan_di()`, karena kelainan lain dikenali dari kata kunci yang bukan milik 6 kelompok lain (67 jenis).
- Katarak, Retinopati hipertensi, dan Miopia patologis cukup menjadi object dari class induk, karena tidak butuh perilaku khusus.
- **Polimorfisme**: `penyakit.tampilkan_info()` dipanggil dengan perintah yang sama, tetapi hasilnya mengikuti class object-nya (contoh pasien ID 188).
- Deteksi kata kunci cocok 100% dengan penanda dokter untuk 3.358 pasien.

Kode yang ditunjukkan & fungsi setiap baris

retinopati_diabetik.py — baris 9–9 (slide 29)

```
 9  class RetinopatiDiabetik(PenyakitMata):
```

Baris	Kode	Fungsinya
9	`class RetinopatiDiabetik(PenyakitMata):`	Membuat class `RetinopatiDiabetik` yang mewarisi `PenyakitMata`.

retinopati_diabetik.py — baris 13–18 (slide 29)

```
13      TINGKAT = (
14          ("proliferative diabetic retinopathy", "proliferatif"),
15          ("severe nonproliferative retinopathy", "berat"),
16          ("moderate non proliferative retinopathy", "sedang"),
17          ("mild nonproliferative retinopathy", "ringan"),
18      )
```

Baris	Kode	Fungsinya
13	`TINGKAT = (`	Konstanta `TINGKAT`: pasangan (kata kunci dokter, nama tingkat), diurutkan dari yang paling berat.
14	`("proliferative diabetic retinopathy", "proliferatif"),`	Tingkat paling berat: proliferatif.
15	`("severe nonproliferative retinopathy", "berat"),`	Tingkat berat.
16	`("moderate non proliferative retinopathy", "sedang"),`	Tingkat sedang (ditulis dokter dengan spasi: 'non proliferative').
17	`("mild nonproliferative retinopathy", "ringan"),`	Tingkat ringan.
18	`)`	Penutup tanda kurung yang dibuka di atasnya.

retinopati_diabetik.py — baris 27–35 (slide 29)

```
27      def tingkat_keparahan(self, pemeriksaan):
28          """Tingkat retinopati diabetik di satu mata (dicari dari yang paling berat)."""
29          if not self.ditemukan_di(pemeriksaan):
30              return "-"
31          for kunci, tingkat in RetinopatiDiabetik.TINGKAT:
32              for kata in pemeriksaan.daftar_kata_kunci():
33                  if kunci in kata:
34                      return tingkat
35          return "tidak disebutkan"
```

Baris	Kode	Fungsinya
27	`def tingkat_keparahan(self, pemeriksaan):`	Method khusus: membaca tingkat retinopati diabetik di satu mata.
28	`"""Tingkat retinopati diabetik di satu mata (dicari dari yang paling berat)."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
29	`if not self.ditemukan_di(pemeriksaan):`	Memakai method warisan `ditemukan_di()`; jika mata ini tidak punya retinopati diabetik …
30	`return "-"`	… kembalikan tanda '-'.
31	`for kunci, tingkat in RetinopatiDiabetik.TINGKAT:`	Mengulang tingkat, mulai dari yang paling berat.
32	`for kata in pemeriksaan.daftar_kata_kunci():`	Mengulang setiap kata kunci diagnosis mata ini.
33	`if kunci in kata:`	Jika kata kunci tingkat itu ditemukan …
34	`return tingkat`	… kembalikan nama tingkatnya (tingkat paling berat yang ditemukan).
35	`return "tidak disebutkan"`	Retinopati diabetik ada, tetapi dokter tidak menyebut tingkatnya.

retinopati_diabetik.py — baris 20–24 (slide 29, kode terkait (dibahas, tidak tampil di slide))

```
20      def __init__(self):
21          # [INHERITANCE] Constructor chaining dengan nilai khusus retinopati diabetik
22          super().__init__("D", "Retinopati diabetik", "retina",
23                           ["nonproliferative retinopathy", "non proliferative retinopathy",
24                            "proliferative diabetic retinopathy", "diabetic retinopathy"])
```

Baris	Kode	Fungsinya
20	`def __init__(self):`	Constructor tanpa parameter, karena data retinopati diabetik selalu sama.
21	`# [INHERITANCE] Constructor chaining dengan nilai khusus retinopati diabetik`	Komentar penjelas (tidak dijalankan Python).
22	`super().__init__("D", "Retinopati diabetik", "retina",`	Constructor chaining: mengisi atribut induk dengan kode D, nama, dan bagian mata (retina) …
23	`["nonproliferative retinopathy", "non proliferative retinopathy",`	… serta list kata kunci pengenalnya …
24	`"proliferative diabetic retinopathy", "diabetic retinopathy"])`	… (lanjutan list kata kunci).

retinopati_diabetik.py — baris 37–42 (slide 29, kode terkait (dibahas, tidak tampil di slide))

```
37      # ---------- [INHERITANCE] Method overriding ----------
38      def tampilkan_info(self, mata_kiri, mata_kanan):
39          """Override: tambahkan tingkat keparahan di setiap mata."""
40          super().tampilkan_info(mata_kiri, mata_kanan)
41          print(f"      tingkat: kiri {self.tingkat_keparahan(mata_kiri)}, "
42                f"kanan {self.tingkat_keparahan(mata_kanan)}")
```

Baris	Kode	Fungsinya
37	`# ---------- [INHERITANCE] Method overriding ----------`	Komentar pembatas bagian kode (tidak dijalankan Python).
38	`def tampilkan_info(self, mata_kiri, mata_kanan):`	Override `tampilkan_info()` milik `PenyakitMata`.

Baris	Kode	Fungsinya
39	`"""Override: tambahkan tingkat keparahan di setiap mata."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
40	`super().tampilkan_info(mata_kiri, mata_kanan)`	Menjalankan versi `tampilkan_info()` milik induk dulu (nama penyakit dan mata yang terkena).
41	`print(f"  tingkat: kiri {self.tingkat_keparahan(mata_kiri)}, "`	Menambahkan tingkat keparahan mata kiri …
42	`f"kanan {self.tingkat_keparahan(mata_kanan)}")`	… dan mata kanan.

pasien_mata.py — baris 119–121 (slide 30)

```
119             for penyakit in self.__daftar_penyakit:
120                 # [POLIMORFISME] Versi tampilkan_info() mengikuti class object penyakitnya
121                 penyakit.tampilkan_info(self.mata_kiri, self.mata_kanan)
```

Baris	Kode	Fungsinya
119	`for penyakit in self.__daftar_penyakit:`	Mengulang setiap penyakit milik pasien.
120	`# [POLIMORFISME] Versi tampilkan_info() mengikuti class object penyakitnya`	Komentar penjelas (tidak dijalankan Python).
121	`penyakit.tampilkan_info(self.mata_kiri, self.mata_kanan)`	**Polimorfisme**: perintahnya sama, tetapi versi `tampilkan_info()` yang berjalan mengikuti class object penyakitnya.

kelainan_lain.py — baris 47–50 (slide 30)

```
47      # ---------- [POLIMORFISME] Method overriding ----------
48      def ditemukan_di(self, pemeriksaan):
49          """Override: kelainan lain ditemukan jika daftar_kelainan() tidak kosong."""
50          return len(self.daftar_kelainan(pemeriksaan)) > 0
```

Baris	Kode	Fungsinya
47	`# ---------- [POLIMORFISME] Method overriding ----------`	Komentar pembatas bagian kode (tidak dijalankan Python).
48	`def ditemukan_di(self, pemeriksaan):`	**Override** `ditemukan_di()` milik induk (polimorfisme), karena cara mengenali kelainan lain berbeda.
49	`"""Override: kelainan lain ditemukan jika daftar_kelainan() tidak kosong."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
50	`return len(self.daftar_kelainan(pemeriksaan)) > 0`	Kelainan lain dianggap ditemukan jika `daftar_kelainan()` tidak kosong.

Output program yang ditunjukkan

Menu 2 · pasien ID 188 (slide 30)

```
Penyakit         : 3 kelompok
  - Retinopati diabetik (retina): mata kiri
      tingkat: kiri proliferatif, kanan -
  - Katarak (lensa mata): mata kanan
  - Kelainan lain (bervariasi): mata kiri
      jenis: laser spot
```

> **Kalimat penutup:** “Selanjutnya, Andi akan menjelaskan program utama.” Naskah lengkap ada di **Naskah Presentasi Kelompok 3**.

Persiapan tanya jawab

T **Apa itu polimorfisme?**

J Satu perintah yang sama menghasilkan perilaku berbeda sesuai class object-nya. Di project kami, `penyakit.tampilkan_info()` menampilkan tingkat untuk retinopati diabetik, jenis untuk kelainan lain, dan versi dasar untuk katarak.

T **Kenapa katarak tidak dibuat class sendiri?**

J Karena data katarak tidak punya informasi tambahan seperti tingkat atau jenis. Membuat class tanpa perilaku khusus hanya menjadi class kosong.

T **Bagaimana program tahu sebuah penyakit ada di satu mata?**

J Method `ditemukan_di()` mencocokkan kata kunci penyakit dengan daftar kata kunci diagnosis mata itu. Hasilnya lalu divalidasi silang dengan penanda dokter.

T **Kenapa tingkat keparahan dicari dari yang paling berat?**

J Satu mata bisa punya lebih dari satu kata kunci. Urutan dari yang paling berat memastikan tingkat tertinggi yang dilaporkan.

BAGIAN 8 | ANDI REZKY NABILA

Program Utama: Membaca CSV & Menu

Slide 31–34 • **Durasi** ± 4 menit • **Konsep** Program utama • **Fokus** main.py: muat_data(), penggabungan baris, try-except, menu interaktif

Inti yang disampaikan

- `main.py` berisi konstanta (`LOKASI_DATA`, `JUMLAH_DITAMPILKAN`, `KOLOM_PASIEN`), fungsi pembantu, `muat_data()`, `demo_validasi()`, lima fungsi menu, `jalankan_menu()`, dan `main()`.
- `muat_data()` membuka file dengan encoding `utf-8-sig`, membaca dengan `csv.DictReader(file, restval="")`, dan mengecek kolom wajib di header lebih dulu.
- Satu pasien bisa muncul di dua baris (satu per foto). Baris kedua harus sama isinya, lalu digabung: 3.034 baris digabung.
- Setiap baris diproses di dalam `try`. Jika ada `ValueError`, nomor barisnya dicatat (`pembaca.line_num`), peringatan dicetak, dan program tetap berjalan.
- Pengujian file rusak: 9 jenis kerusakan, semuanya dilewati dengan pesan yang tepat. File tanpa kolom wajib dan file yang tidak ada juga ditangani.
- Menu berulang dengan `while True` sampai pilihan 6 (`break`). Pilihan di luar 1–6 mendapat pesan.
- Tambah pasien: ID otomatis (ID terbesar + 1), data dicek oleh setter yang sama, dan pasien baru hanya disimpan di list selama program berjalan (CSV asli tidak diubah).

Kode yang ditunjukkan & fungsi setiap baris

main.py — baris 109–122 (slide 32)

```
109  def muat_data(lokasi_file, daftar_jenis):
110      """Membaca CSV, menggabungkan baris milik pasien yang sama, dan mencatat baris tidak
     valid."""
111      daftar_pasien = []
112      isi_per_id = {}        # ID -> isi kolom pasien dari baris pertama yang valid
113      baris_dilewati = []    # nomor baris CSV yang tidak valid
114      baris_digabung = 0     # baris foto mata kedua dari pasien yang sama
115      # utf-8-sig: tetap aman jika CSV pernah disimpan ulang lewat Excel (ada BOM)
116      with open(lokasi_file, mode="r", encoding="utf-8-sig", newline="") as file:
117          # restval="": kolom yang hilang dianggap teks kosong, jadi ikut ditolak validasi
118          pembaca = csv.DictReader(file, restval="")
119          for kolom in KOLOM_PASIEN:
120              if kolom not in (pembaca.fieldnames or []):
121                  raise ValueError(f"Kolom '{kolom}' tidak ada di file CSV")
122          for baris in pembaca:
```

Baris	Kode	Fungsinya
109	`def muat_data(lokasi_file, daftar_jenis):`	Fungsi untuk membaca seluruh file CSV menjadi list object.
110	`"""Membaca CSV, menggabungkan baris milik pasien yang sama, dan mencatat baris tidak valid."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
111	`daftar_pasien = []`	List untuk menampung semua object `PasienMata`.
112	`isi_per_id = {}  # ID -> isi kolom pasien dari baris pertama yang valid`	Dictionary ID → isi kolom pasien dari baris pertama, untuk mengenali baris kedua pasien yang sama.
113	`baris_dilewati = []  # nomor baris CSV yang tidak valid`	List nomor baris CSV yang tidak valid.
114	`baris_digabung = 0  # baris foto mata kedua dari pasien yang sama`	Penghitung baris foto kedua yang digabung.
115	`# utf-8-sig: tetap aman jika CSV pernah disimpan ulang lewat Excel (ada BOM)`	Komentar penjelas (tidak dijalankan Python).

Baris	Kode	Fungsinya
116	`with open(lokasi_file, mode="r", encoding="utf-8-sig", newline="") as file:`	Membuka file untuk dibaca; `with` menutup file otomatis. `utf-8-sig` aman untuk file yang pernah disimpan Excel.
117	`# restval="": kolom yang hilang dianggap teks kosong, jadi ikut ditolak validasi`	Komentar penjelas (tidak dijalankan Python).
118	`pembaca = csv.DictReader(file, restval="")`	Membaca CSV sebagai dictionary per baris (kolom diakses dengan namanya). `restval=""` mengisi kolom yang hilang dengan teks kosong.
119	`for kolom in KOLOM_PASIEN:`	Mengecek setiap kolom wajib …
120	`if kolom not in (pembaca.fieldnames or []):`	… apakah ada di header file (`fieldnames`).
121	`raise ValueError(f"Kolom '{kolom}' tidak ada di file CSV")`	Kolom wajib tidak ada: hentikan pembacaan dengan `ValueError` (ditangkap di `main()`).
122	`for baris in pembaca:`	Mengulang setiap baris data di file.

main.py — baris 123–137 (slide 33)

```
123             try:
124                 isi = tuple(baris[kolom] for kolom in KOLOM_PASIEN)
125                 id_teks = baris["ID"].strip()
126                 if id_teks in isi_per_id:
127                     # Baris foto mata kedua: isinya harus sama dengan baris pertama
128                     if isi != isi_per_id[id_teks]:
129                         raise ValueError(f"Data pasien ID {id_teks} berbeda dengan baris sebelumnya")
130                     baris_digabung += 1
131                     continue
132                 daftar_pasien.append(buat_pasien(baris, daftar_jenis))
133                 isi_per_id[id_teks] = isi
134             except ValueError as error:
135                 # Baris yang melanggar aturan dilewati dan dicatat, program tetap berjalan
136                 baris_dilewati.append(pembaca.line_num)
137                 print(f"[PERINGATAN] Baris {pembaca.line_num} dilewati: {error}")
```

Baris	Kode	Fungsinya
123	`try:`	`try`: menjalankan kode di bawahnya sambil berjaga-jaga jika muncul error.
124	`isi = tuple(baris[kolom] for kolom in KOLOM_PASIEN)`	Mengambil isi 15 kolom pasien sebagai tuple, untuk dibandingkan dengan baris lain.
125	`id_teks = baris["ID"].strip()`	Mengambil ID pasien (masih berupa teks).
126	`if id_teks in isi_per_id:`	Jika ID ini sudah pernah dibaca, ini baris foto kedua pasien yang sama:
127	`# Baris foto mata kedua: isinya harus sama dengan baris pertama`	Komentar penjelas (tidak dijalankan Python).
128	`if isi != isi_per_id[id_teks]:`	isinya harus sama dengan baris pertama; jika berbeda …
129	`raise ValueError(f"Data pasien ID {id_teks} berbeda dengan baris sebelumnya")`	… baris ditolak dengan `ValueError`.
130	`baris_digabung += 1`	Jika sama, hitung sebagai baris yang digabung …
131	`continue`	Lewati sisa perulangan, lanjut ke data berikutnya.
132	`daftar_pasien.append(buat_pasien(baris, daftar_jenis))`	Pasien baru: ubah baris menjadi object lewat `buat_pasien()`, lalu simpan ke list dengan `append`.
133	`isi_per_id[id_teks] = isi`	Mengingat isi baris ini untuk ID tersebut.
134	`except ValueError as error:`	Menangkap `ValueError` dari blok `try`; pesannya disimpan di `error`, dan program tidak berhenti.
135	`# Baris yang melanggar aturan dilewati dan dicatat, program tetap berjalan`	Komentar penjelas (tidak dijalankan Python).
136	`baris_dilewati.append(pembaca.line_num)`	Mencatat nomor baris CSV yang sedang dibaca (`line_num`).
137	`print(f"[PERINGATAN] Baris {pembaca.line_num} dilewati: {error}")`	Mencetak peringatan berisi nomor baris dan alasannya.

main.py — baris 224–224 (slide 34)

```
224     id_baru = max(pasien.id_pasien for pasien in daftar_pasien) + 1
```

Baris	Kode	Fungsinya
224	`id_baru = max(pasien.id_pasien for pasien in daftar_pasien) + 1`	ID baru = ID terbesar + 1, supaya tidak ada ID ganda.

main.py — baris 228–242 (slide 34)

```
228     try:
229         umur = ke_bilangan_bulat(input("Umur                          : "), "Umur")
230         jenis_kelamin = input("Jenis kelamin (Male/Female)  : ").strip()
231         mata_kiri = PemeriksaanMata("kiri", f"{id_baru}_left.jpg", input("Diagnosis mata kiri
    : "))
232         mata_kanan = PemeriksaanMata("kanan", f"{id_baru}_right.jpg", input("Diagnosis mata
    kanan         : "))
233         daftar_penyakit = deteksi_penyakit(mata_kiri, mata_kanan, daftar_jenis)
234         kode = ""
235         for penyakit in daftar_penyakit:
236             kode += penyakit.kode
237         # [ENKAPSULASI] Data baru diperiksa oleh setter yang sama dengan data CSV
238         pasien = PasienMata(id_baru, umur, jenis_kelamin, mata_kiri, mata_kanan, kode or "N",
    daftar_penyakit)
239     except ValueError as error:
240         print(f"Data tidak valid: {error}")
241         print("Pasien baru tidak ditambahkan.")
242         return
```

Baris	Kode	Fungsinya
228	`try:`	`try`: menjalankan kode di bawahnya sambil berjaga-jaga jika muncul error.
229	`umur = ke_bilangan_bulat(input("Umur : "), "Umur")`	Meminta umur, lalu mengubahnya menjadi bilangan bulat.
230	`jenis_kelamin = input("Jenis kelamin (Male/Female)  : ").strip()`	Meminta jenis kelamin (spasi di tepi dibuang).
231	`mata_kiri = PemeriksaanMata("kiri", f"{id_baru}_left.jpg", input("Diagnosis mata kiri : "))`	Meminta diagnosis mata kiri dan langsung membuat object `PemeriksaanMata`; nama foto dibuat dari ID baru.
232	`mata_kanan = PemeriksaanMata("kanan", f"{id_baru}_right.jpg", input("Diagnosis mata kanan : "))`	Sama untuk mata kanan.
233	`daftar_penyakit = deteksi_penyakit(mata_kiri, mata_kanan, daftar_jenis)`	Mendeteksi penyakit dari kata kunci diagnosis.
234	`kode = ""`	Teks kosong untuk menampung kode penyakit.
235	`for penyakit in daftar_penyakit:`	Mengulang penyakit yang terdeteksi …
236	`kode += penyakit.kode`	… dan menggabungkan kodenya.
237	`# [ENKAPSULASI] Data baru diperiksa oleh setter yang sama dengan data CSV`	Komentar penjelas (tidak dijalankan Python).
238	`pasien = PasienMata(id_baru, umur, jenis_kelamin, mata_kiri, mata_kanan, kode or "N", daftar_penyakit)`	Membuat object pasien; `kode or "N"` berarti jika tidak ada penyakit, kodenya N (normal). Semua setter ikut memeriksa.
239	`except ValueError as error:`	Menangkap `ValueError` dari blok `try`; pesannya disimpan di `error`, dan program tidak berhenti.
240	`print(f"Data tidak valid: {error}")`	Menampilkan alasan data ditolak.
241	`print("Pasien baru tidak ditambahkan.")`	Memberi tahu pasien tidak jadi ditambahkan.
242	`return`	Keluar dari fungsi lebih awal.

main.py — baris 292–317 (slide 34, kode terkait (dibahas, tidak tampil di slide))

```
292 def jalankan_menu(daftar_pasien, daftar_jenis):
293     """Menampilkan menu berulang-ulang sampai pengguna memilih Keluar."""
294     while True:
295         cetak_judul("MENU")
296         print(f"1. Lihat {JUMLAH_DITAMPILKAN} data pertama")
297         print("2. Cari pasien berdasarkan ID")
298         print("3. Cari pasien berdasarkan penyakit")
299         print("4. Tambah pasien baru")
300         print("5. Tampilkan ringkasan")
301         print("6. Keluar")
302         pilihan = input("Pilih menu (1-6): ").strip()
303         if pilihan == "1":
304             menu_lihat_data(daftar_pasien)
305         elif pilihan == "2":
306             menu_cari_id(daftar_pasien)
307         elif pilihan == "3":
308             menu_cari_penyakit(daftar_pasien, daftar_jenis)
309         elif pilihan == "4":
310             menu_tambah_pasien(daftar_pasien, daftar_jenis)
311         elif pilihan == "5":
312             menu_ringkasan(daftar_pasien, daftar_jenis)
313         elif pilihan == "6":
314             print("Terima kasih. Program selesai.")
315             break
316         else:
317             print("Pilihan tidak tersedia. Masukkan angka 1 sampai 6.")
```

Baris	Kode	Fungsinya
292	`def jalankan_menu(daftar_pasien, daftar_jenis):`	Fungsi menu utama yang berulang sampai pengguna memilih Keluar.
293	`"""Menampilkan menu berulang-ulang sampai pengguna memilih Keluar."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
294	`while True:`	Perulangan tanpa batas; berhenti hanya lewat `break`.
295	`cetak_judul("MENU")`	Mencetak judul menu.
296	`print(f"1. Lihat {JUMLAH_DITAMPILKAN} data pertama")`	Mencetak pilihan 1.
297	`print("2. Cari pasien berdasarkan ID")`	Mencetak pilihan 2.
298	`print("3. Cari pasien berdasarkan penyakit")`	Mencetak pilihan 3.
299	`print("4. Tambah pasien baru")`	Mencetak pilihan 4.
300	`print("5. Tampilkan ringkasan")`	Mencetak pilihan 5.
301	`print("6. Keluar")`	Mencetak pilihan 6.
302	`pilihan = input("Pilih menu (1-6): ").strip()`	Membaca pilihan pengguna dari keyboard.
303	`if pilihan == "1":`	Pilihan 1 …
304	`menu_lihat_data(daftar_pasien)`	… menjalankan menu lihat data.
305	`elif pilihan == "2":`	Pilihan 2 …
306	`menu_cari_id(daftar_pasien)`	… menjalankan menu cari ID.
307	`elif pilihan == "3":`	Pilihan 3 …
308	`menu_cari_penyakit(daftar_pasien, daftar_jenis)`	… menjalankan menu cari penyakit.
309	`elif pilihan == "4":`	Pilihan 4 …
310	`menu_tambah_pasien(daftar_pasien, daftar_jenis)`	… menjalankan menu tambah pasien.
311	`elif pilihan == "5":`	Pilihan 5 …
312	`menu_ringkasan(daftar_pasien, daftar_jenis)`	… menjalankan menu ringkasan.
313	`elif pilihan == "6":`	Pilihan 6 …
314	`print("Terima kasih. Program selesai.")`	… mencetak salam penutup …
315	`break`	… lalu keluar dari perulangan `while`.

Baris	Kode	Fungsinya
316	`else:`	Jika kondisi sebelumnya tidak terpenuhi, bagian di bawahnya yang dijalankan.
317	`print("Pilihan tidak tersedia. Masukkan angka 1 sampai 6.")`	Pilihan lain: cetak pesan, lalu menu tampil lagi.

> **Kalimat penutup:** “Selanjutnya, Alfira akan menunjukkan hasil program dan kesimpulan kami.” Naskah lengkap ada di **Naskah Presentasi Kelompok 3**.

Persiapan tanya jawab

T **Apa fungsi try-except di muat_data()?**

J Menangkap ValueError dari baris yang melanggar aturan, supaya satu baris salah tidak menghentikan pembacaan seluruh file. Nomor barisnya dicatat agar bisa diperiksa.

T **Kenapa memakai DictReader, bukan csv.reader biasa?**

J DictReader membaca setiap baris sebagai dictionary, jadi kolom diambil dengan namanya, misalnya `baris['Patient Age']`. Lebih mudah dibaca dan tidak tergantung urutan kolom.

T **Kenapa pasien baru tidak disimpan ke file CSV?**

J Disengaja, agar data asli tetap utuh. Pasien baru tersimpan di list selama program berjalan.

T **Apa arti if __name__ == "__main__"?**

J Fungsi `main()` hanya dijalankan jika file dijalankan langsung dengan `python main.py`, bukan saat file di-import oleh file lain.

BAGIAN 9 | ALFIRA YUNITA SYAM

Hasil Program, Demo Menu & Kesimpulan

Slide 35–41 • **Durasi** ± 5 menit • **Konsep** Hasil & kesimpulan • **Fokus** Output nyata program, ringkasan data, kesimpulan, dan catatan

Inti yang disampaikan

- Output awal: 3.358 object PasienMata dimuat, 3.034 baris digabung, 0 baris dilewati.
- Menu 1 menampilkan 10 pasien pertama; kolom penyakit berasal dari method `nama_penyakit()`.
- Menu 2 mencari ID: ID 188 tampil lengkap (3 kelompok penyakit), ID 12 tidak ditemukan, dan input `abc` mendapat pesan harus angka.
- Menu 3 mencari penyakit: glaukoma 206 pasien, lengkap dengan mata yang terkena.
- Menu 4 menambah pasien ID 4785 (retinopati diabetik dan katarak; total menjadi 3.359), sedangkan data dengan jenis kelamin 'Laki-laki' ditolak.
- Menu 5 ringkasan: penyakit terbanyak retinopati diabetik (1.105 pasien); rata-rata umur tertinggi katarak (66,4 tahun).
- Kesimpulan: ketiga konsep OOP diterapkan bersama komposisi dan polimorfisme. Catatan: data hanya memuat penyakit yang terlihat di foto fundus, dan program ini pengelola data, bukan alat diagnosis.

Output program yang ditunjukkan

python main.py (slide 36)

```
======================================================================
SISTEM MANAJEMEN DATA PASIEN PENYAKIT MATA
======================================================================
File data          : full_df.csv
Pasien dimuat      : 3358 object PasienMata
Baris digabung     : 3034 (foto mata kedua dari pasien yang sama)
Baris dilewati     : 0
…
======================================================================
MENU
======================================================================
1. Lihat 10 data pertama
2. Cari pasien berdasarkan ID
3. Cari pasien berdasarkan penyakit
4. Tambah pasien baru
5. Tampilkan ringkasan
6. Keluar
Pilih menu (1-6): 1
```

Menu 1 · lihat 10 data pertama (slide 36)

```
======================================================================
10 DATA PERTAMA (dari 3358 pasien)
======================================================================
ID    Umur  Kelamin  Penyakit
0     69    Female   Katarak
1     57    Male     Normal
2     42    Male     Retinopati diabetik, Kelainan lain
4     53    Male     Retinopati diabetik, Kelainan lain
5     50    Female   Retinopati diabetik
6     60    Male     Retinopati diabetik, Kelainan lain
7     60    Female   Retinopati diabetik, Kelainan lain
8     59    Male     Normal
9     54    Male     Kelainan lain
10    70    Male     Kelainan lain
```

Menu 2 · cari berdasarkan ID (slide 37)

```
Masukkan ID pasien: 12
Pasien dengan ID 12 tidak ditemukan.
…
Masukkan ID pasien: abc
ID pasien harus berupa angka.
```

Menu 3 · cari berdasarkan penyakit (slide 37)

```
1. Retinopati diabetik
2. Glaukoma
3. Katarak
4. Degenerasi makula
5. Retinopati hipertensi
6. Miopia patologis
7. Kelainan lain
Pilih penyakit (1-7): 2

Glaukoma: 206 pasien. 10 pertama:
ID    Umur  Kelamin  Mata terkena
43    35    Male     mata kanan
95    46    Male     mata kiri
153   79    Male     mata kiri
167   71    Male     kedua mata
178   54    Male     mata kiri
238   70    Male     mata kanan
247   49    Male     kedua mata
365   60    Female   kedua mata
583   54    Male     kedua mata
625   72    Female   mata kiri
```

Menu 4 · tambah pasien baru (slide 38)

```
ID pasien baru (otomatis)    : 4785
Tulis diagnosis dengan istilah dokter seperti di data, pisahkan dengan koma.
Contoh: normal fundus / cataract / glaucoma / mild nonproliferative retinopathy
Umur                         : 63
Jenis kelamin (Male/Female)  : Female
Diagnosis mata kiri          : cataract
Diagnosis mata kanan         : moderate non proliferative retinopathy
Pasien ID 4785 berhasil ditambahkan (Retinopati diabetik, Katarak). Jumlah pasien sekarang: 3359
```

Menu 4 · data salah (slide 38)

```
Umur                         : 45
Jenis kelamin (Male/Female)  : Laki-laki
Diagnosis mata kiri          : normal fundus
Diagnosis mata kanan         : normal fundus
Data tidak valid: Jenis kelamin harus 'Male' atau 'Female', bukan 'Laki-laki'
Pasien baru tidak ditambahkan.
```

Menu 2 · cari pasien 4785 (slide 38)

```
Masukkan ID pasien: 4785
ID Pasien        : 4785
Umur             : 63 tahun
Jenis Kelamin    : Female
Mata kiri        : cataract
Mata kanan       : moderate non proliferative retinopathy
Penyakit         : 2 kelompok
  - Retinopati diabetik (retina): mata kanan
      tingkat: kiri -, kanan sedang
  - Katarak (lensa mata): mata kiri
```

Menu 5 · ringkasan (slide 39)

```
Jumlah pasien             : 3358
Laki-laki / Perempuan     : 1800 / 1558
Rata-rata umur            : 57.9 tahun
Pasien normal             : 1080
Pasien > 1 kelompok       : 556

Penyakit                Pasien  Rata umur  Kedua mata
Retinopati diabetik       1105       56.4         659
Glaukoma                   206       62.6         110
Katarak                    208       66.4          99
Degenerasi makula          163       60.9         116
Retinopati hipertensi      103       56.2          90
Miopia patologis           171       57.0          91
Kelainan lain              905       59.1         173

Penyakit terbanyak        : Retinopati diabetik (1105 pasien)
```

> **Kalimat penutup:** "Sekian presentasi dari Kelompok 3." Naskah lengkap ada di **Naskah Presentasi Kelompok 3**.

Persiapan tanya jawab

T **Apakah output di slide asli?**

J Ya. Semua output disalin otomatis dari hasil menjalankan `python main.py`, tidak diketik ulang.

T **Kenapa ID 12 tidak ada?**

J Nomor ID di data tidak berurutan. ID-nya 0–4784, tetapi pasiennya hanya 3.358, jadi ada 1.427 nomor yang kosong.

T **Apakah program ini bisa mendiagnosis penyakit mata?**

J Tidak. Diagnosis tetap dari dokter. Program mengelola, memeriksa, dan merangkum data diagnosis itu.

T **Apa kelebihan program ini?**

J Data nyata, setiap nilai divalidasi, baris salah dilewati tanpa menghentikan program, penyakit dikenali per mata, dan strukturnya mudah dikembangkan untuk penyakit baru.